

Reviewer Pack — Minimal Monomial Ideals Rank and Communication Complexity Ra...

Entry f191578e744b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 22:33:43 UTC

Minimal Monomial Ideals Rank and Communication Complexity Rank Variance Bound

Entry ID: f191578e744b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 22:33:43 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monomial Ideals) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given boolean formula with n variables, the minimal rank of its associated monomial ideal is linearly correlated with its communication complexity rank variance, such that $r(m_{\text{ideal}}) \leq k * O(n^{(1/2)})$ for some constant k .

Rationale (proposer's reasoning):

Monomial ideals can provide structural information about the polynomial equations defining a boolean formula. By studying their ranks, we may uncover a connection to communication complexity, potentially leading to new insights into lower bounds and hardness of approximation for communication problems. The logarithmic relationship with n suggests that this is a non-trivial connection between two fields.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 22d78fbec8758700

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between monomial ideal ranks and communication complexity rank variance across all seeds is ≥ 0.5 , otherwise it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "monomial ideal" AND "communication complexity rank" AND "rank variance" - "algebraic geometry" AND "matrix rank" AND "communication complexity" - "minimal monomial ideal rank" AND "linear correlation" AND "communication complexity rank"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        return ''.join(random.choice('01') for _ in range(2**n))

    def monomial_ideal_rank(formula):
        # Simplified version of monomial ideal rank calculation
        return len(set(int(bit) for bit in formula if bit == '1'))

    def communication_complexity_rank_variance(formula):
        # Simplified version of communication complexity rank variance calculation
        ones = formula.count('1')
        zeros = formula.count('0')
        return (ones - zeros)**2 / len(formula)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        formula = generate_boolean_formula(n)
        rank = monomial_ideal_rank(formula)
        variance = communication_complexity_rank_variance(formula)
        results.append((rank, variance))

    if not results:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
```

```

        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

    ranks = [r for r, _ in results]
    variances = [v for _, v in results]

    mean_rank = sum(ranks) / len(ranks)
    mean_variance = sum(variances) / len(variances)

    covariance = sum((r - mean_rank) * (v - mean_variance) for r, v in results) / len(results)
    variance_of_ranks = sum((r - mean_rank)**2 for r in ranks) / len(ranks)

    correlation_coefficient = covariance / math.sqrt(variance_of_ranks * sum((v - mean_variance)**2 for v in variances) / len(variances))

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if not results:
        print("RESULT: INCONCLUSIVE no_results")
    else:
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
        elif any(not r["conjecture_holds"] for r in results):
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
        else:
            print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the Pearson correlation coefficient could not be calculated to determine if the conjecture is supported or falsified. | next: Run the test again with increased time limits and ensure it completes successfully to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18547
2	preregistration	ollama_remote	glm4:latest	0	0	8901
3	novelty	ollama_remote	glm4:latest	0	0	8335
4	novelty	ollama_remote	glm4:latest	0	0	9342
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15460
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16433
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12015
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10650
9	judge	ollama_remote	glm4:latest	0	0	29539

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129222 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f191578e744b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f191578e744b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f191578e744b.tar.gz` (if generated)